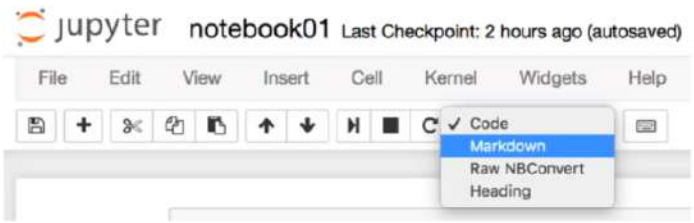
Lab 30_1: Basic Example with Jupyter Notebook

1.include explanatory text in your notebook



2. Once switched the type to *Markdown* you can start typing in markdown code:



logictronix

Lab 30_1: Basic Example with Jupyter Notebook

- Check Python Version on Jupyter Notebook

```
In [11]: >>> from platform import python_version
         >>> print(python_version())
         3.6.5
```

- If Tensorflow is installed then, checking it's version:

```
import tensorflow as tf

print tf.VERSION
```

logictronix

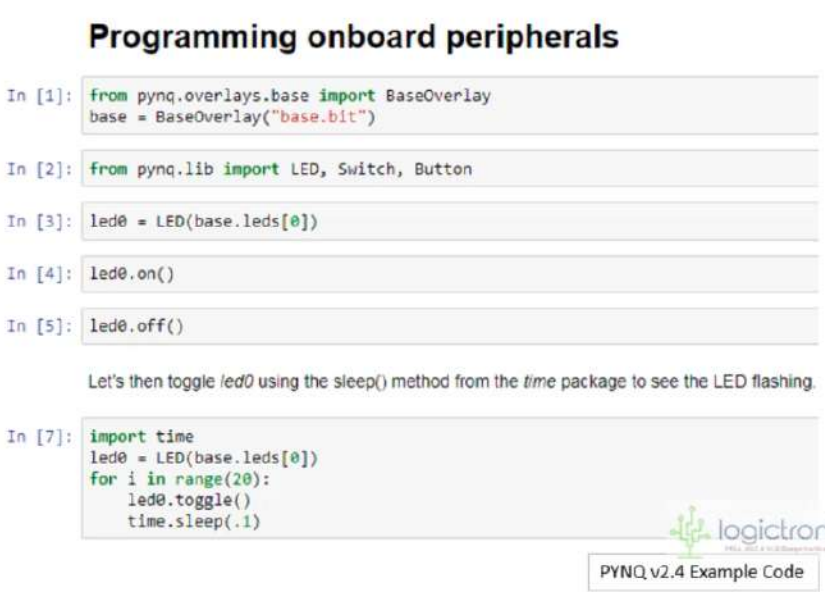
Pynq libraries for the button and led modules

- Sleep- used to add delay in the execution of the program
- Rgbled_position-
- On- LED on
- Off- LED off
- Buttons- controls the onboard push button
- Read- read the current values of the button
- Toggle-This method checks the selected elements for visibility.
- Wait_for_value- wait for the button to be pressed or relaesed



LAB 30_2: GPIO Control with PYNQ

- Lets do this basic GPIO example on Jupyter Notebook



Programming onboard peripherals

In [1]:
```
from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
```

In [2]:
```
from pynq.lib import LED, Switch, Button
```

In [3]:
```
led0 = LED(base.leds[0])
```

In [4]:
```
led0.on()
```

In [5]:
```
led0.off()
```

Let's then toggle *led0* using the sleep() method from the *time* package to see the LED flashing.

In [7]:
```
import time
led0 = LED(base.leds[0])
for i in range(20):
    led0.toggle()
    time.sleep(.1)
```

PYNQ v2.4 Example Code

Pynq Audio library

- Record()- Record data from audio controller to the audio buffer
- Save()- save audio buffer content to the file
- Load()- load files into the internal audio buffer
- Play()- play audio buffer via audio jack

PYNQ v2.4 Example Code for Audio

```
from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
audio = base.audio
```

```
# Record a sample
audio.record(4)
# Save recorded sample
audio.save("Recording_1.pdm")
```

```
# Play recorded sample
audio.play()
```

Load and play

Load a sample and play the loaded sample.

```
audio.load("/home/xilinx/jupyter_notebooks/base/audio/Recording_1.pdm")
audio.play()
```

logictronix

Pynq Video Library

- Configure()- Configure the pipeline to use the specified pixel format.
- Start()- To start the video transfer through the hdmi in
- Tie()-The monitor should turn on and show a blank screen. To pass the image data through we can tie the output to the input. The tie will last until we send something else to be displayed.
- Numframes-Returns a frame of the appropriate size for the video mode.
- writeframe-write the frame to the video output
- readframe- Read the video frame
- Cvtcolour()-Converts an image from one color space to another

PYNQ v2.1 Example Code for Video

```
from pynq import Overlay
from pynq.drivers.video import HDMI

# Download bitstream
Overlay("base.bit").download()

# Initialize HDMI as an input device
hdmi_in = HDMI('in')
```

```
hdmi_in.start()

width = hdmi_in.frame_width()
height = hdmi_in.frame_height()
print('HDMI is capturing a video source of resolution {}x{}'\
      .format(width,height))
```

logictronix

Extra thing to Know → Python Indentation

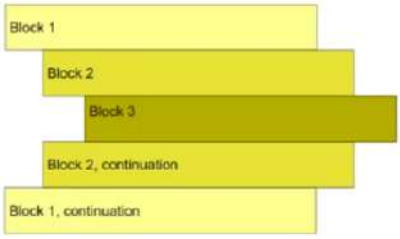
- **Python** uses **indentation** instead of braces to structure its programs.

Python Indentation

```
In [68]: if (switch.read()==1):
             led0.on()
         else:
                 led0.off()
```

```
In [70]: if (switch.read()==1):
             #led0.on()
             led0.on()
         else:
                 led0.off()
```

```
  File "<ipython-input-70-a0c130a4b717>", line 4
    else:
    ^
IndentationError: unindent does not match any outer indentation level
```



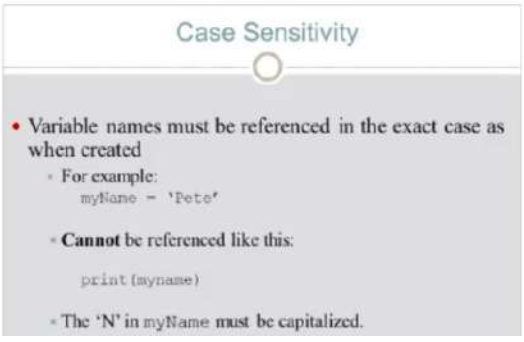
logictronix

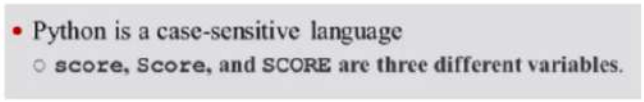
Another thing to Know

- **Identifiers**

  Python is case sensitive. These are rules for creating an identifier:

  - Must start with a letter or underscore (_)
  - Can be followed by any number of letters, digits, or underscores.
  - Cannot be a reserved word.





logictronix

Topic 1: Conditional Statements in Python

- If-else Statement

```
In [7]: a = 10
        b = 20
        if a == b:
            print('yes')
        else:
            print('no')

        no

In [8]: a = 10
        b = 10
        if a == b:
            print('yes')
        else:
            print('no')

        yes
```

- If-else-if Statement

```
In [4]: a = 10
        b = 11
        c = 10
        if a == b:
            print('first condition is true')
        elif a == c:
            print('second condition is true')
        else:
            print('nothing is true. existence is pain.')

        second condition is true
```

logictronix

Lab 31_1: Basic Example of GPIO with if-else statement with Jupyter Notebook



GPIO with Conditional Statement

In [35]:
```
from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
```

In [36]:
```
from pynq.lib import LED, Switch, Button
```

In [37]:
```
led0 = LED(base.leds[0])
```

In [38]:
```
switch= Switch(base.switches[0])
```

In [40]:
```
if (switch.read()==1):
    #led0.on()
    led0.on()
else:
        led0.off()
```

logictronix

Topic 2: Loops in Python

- For Loop

```
In [2]: numbers =[1,2,3,4,5]
        for i in numbers:
            print(i)

        1
        2
        3
        4
        5
```

```
In [9]: numbers = [1, 5, 12, 91, 102]

In [10]: for i in numbers:
             print(i * i)

         1
         25
         144
         8281
         10404
```

```
In [1]: from pynq.overlays.base import BaseOverlay
        base = BaseOverlay("base.bit")

In [2]: from pynq.lib import LED, Switch, Button

In [3]: #turn ON all LED's
        for led in base.leds:
            led.on()
```

Turn ON all LED's

```
In [12]: from pynq.overlays.base import BaseOverlay
         base = BaseOverlay("base.bit")

In [13]: from pynq.lib import LED, Switch, Button
         import time

In [15]: #turn ON all LED's on interval of 1 second
         for led in base.leds:
             led.on()
             time.sleep(1)
             led.off()
```

Turn ON LED's in sequence of 1 second

logictronix

Topic 2: Loops in Python

- While Loop

While Loop Test

```
In [2]: from pynq.overlays.base import BaseOverlay
        base = BaseOverlay("base.bit")

In [3]: from pynq.lib import LED, Switch, Button

In [4]: led0 = LED(base.leds[0])
        switch= Switch(base.switches[0])

In [12]: while(switch.read()==1):
             led0.on()

         led0.off()
```

- While loop [infinite]

Infinite While Loop Test

```
In [2]: from pynq.overlays.base import BaseOverlay
        base = BaseOverlay("base.bit")

In [3]: from pynq.lib import LED, Switch, Button

In [4]: led0 = LED(base.leds[0])
        switch= Switch(base.switches[0])

In [*]: while True:
            if(switch.read()==1):
                led0.on()
            else:
                led0.off()
```

How to Stop Infinite



logictronix

Topic 2: Loops in Python

- While-else loop

```
In [12]: from pynq.overlays.base import BaseOverlay
         base = BaseOverlay("base.bit")

In [13]: from pynq.lib import LED, Switch, Button
         import time

In [17]: led0 = LED(base.leds[0])
         switch= Switch(base.switches[0])

In [19]: while(switch.read()==1):
             led0.on()
         else:
                led0.off()
```

- While-else loop

logictronix

Lab 31_2: GPIO Control with Loops on PYNQ

While If-else-if Statement for GPIO Control

In [1]:
```
from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
```

In [2]:
```
from pynq.lib import LED, Switch, Button
```

In [3]:
```
#Turn ON all LED's
for led in base.leds:
    led.on()
```

Controlling all switches, led and buttons

In [4]:
```
# Set the number of LED, Switches, and Buttons
MAX_LEDS =4
MAX_SWITCHES = 2
MAX_BUTTONS = 4

leds = [LED(base.leds[index]) for index in range(MAX_LEDS)]
switches = [Switch(base.switches[index]) for index in range(MAX_SWITCHES)]
buttons = [Button(base.buttons[index]) for index in range(MAX_BUTTONS)]
```

In [5]:
```
# Create lists for each of the IO component groups
for i in range(MAX_LEDS):
    leds[i] = LED(base.leds[i])
for i in range(MAX_SWITCHES):
    switches[i] = Switch(base.switches[i])
for i in range(MAX_BUTTONS):
```

Changing the corresponding LED as the Switch Value

In [10]:
```
# LEDs start in the off state
for i in range(MAX_LEDS):
    leds[i].off()

# if a slide-switch is on, light the corresponding LED
for i in range(MAX_LEDS):
    if switches[i%2].read():
        leds[i].on()
    else:
        leds[i].off()
```

If the Push Buttons is pressed and following code run, then the corresponding LED will toggle

In [16]:
```
# if a button is depressed, toggle the state of the corresponding LED
for i in range(MAX_LEDS):
    if buttons[i].read():
        leds[i].toggle()
```

logictronix

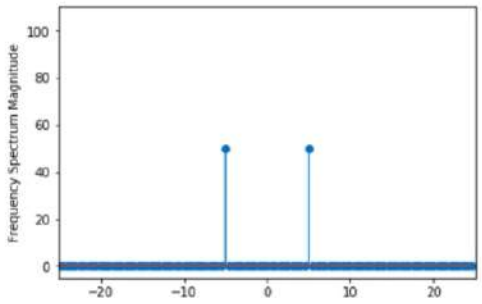
Topic 3: Python Libraries: SciPy

SciPy Library

In [5]:
```
from scipy import fftpack

A = fftpack.fft(a)
frequency = fftpack.fftfreq(len(a)) * fre_samp
figure, axis = plt.subplots()

axis.stem(frequency, np.abs(A))
axis.set_xlabel('Frequency in Hz')
axis.set_ylabel('Frequency Spectrum Magnitude')
axis.set_xlim(-fre_samp / 2, fre_samp/ 2)
axis.set_ylim(-5, 110)
plt.show()
```



logictronix

Topic 3: Python Libraries: SciPy

SciPy for image processing

Image Processing with SciPy – scipy.ndimage

- scipy.ndimage is a submodule of SciPy which is mostly used for performing an image related operation
- ndimage means the "n" dimensional image.
- SciPy Image Processing provides Geometrics transformation (rotate, crop, flip), image filtering (sharp and de nosing), display image, image segmentation, classification and features extraction.
- **MISC Package** in SciPy contains prebuilt images which can be used to perform image manipulation task

SciPy for image processing

In [7]:
```
from scipy import misc
from matplotlib import pyplot as plt
import numpy as np
#get face image of panda from misc package
panda = misc.face()
#plot or show image of face
plt.imshow( panda )
plt.show()
```



logictronix

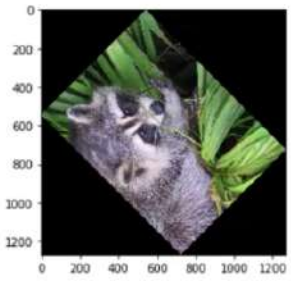
Topic 3: Python Libraries: SciPy

SciPy for image processing

In [8]:
```
#Flip Down using scipy misc.face image
flip_down = np.flipud(misc.face())
plt.imshow(flip_down)
plt.show()
```



In [9]:
```
from scipy import ndimage, misc
from matplotlib import pyplot as plt
panda = misc.face()
#rotatation function of scipy for image - image rotated 135 degree
panda_rotate = ndimage.rotate(panda, 135)
plt.imshow(panda_rotate)
plt.show()
```



logictronix

Topic 3: Python Libraries: NumPy

Numpy

In [11]:
```
import numpy as np
print (np.__version__)
```

```
1.13.3
```

In [13]:
```
#Create a NumPy Array
#Simplest way to create an array in Numpy is to use Python List
myPythonList = [1,9,8,3]
```

In [15]:
```
numpy_array_from_list = np.array(myPythonList)
numpy_array_from_list
```

Out[15]:
```
array([1, 9, 8, 3])
```

logictronix

Topic 3: Python Libraries: MatPlot

MatPlot

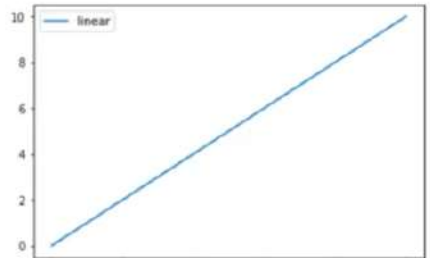
In [23]:
```
import matplotlib.pyplot as plt
import numpy as np

# Prepare the data
x = np.linspace(0, 10, 100)

# Plot the data
plt.plot(x, x, label='linear')

# Add a legend
plt.legend()

# Show the plot
plt.show()
```

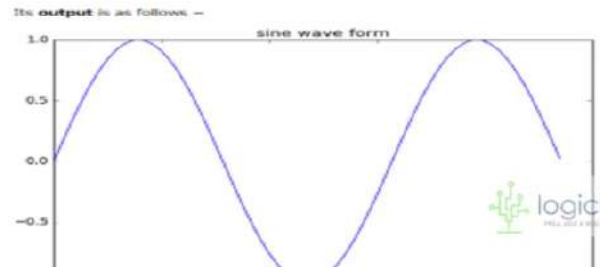



```
import numpy as np
import matplotlib.pyplot as plt

# Compute the x and y coordinates for points on a sine curve
x = np.arange(0, 3 * np.pi, 0.1)
y = np.sin(x)
plt.title("sine wave form")

# Plot the points using matplotlib
plt.plot(x, y)
plt.show()
```

Its **output** is as follows –



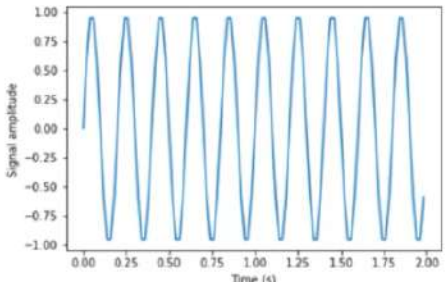
logictronix

Lab 31_3: Using the Lecture 3 libraries

Matplot library

In [4]:
```
%matplotlib inline
from matplotlib import pyplot as plt
import numpy as np

#Frequency in terms of Hertz
fre  = 5
#Sample rate
fre_samp = 50
t = np.linspace(0, 2, 2 * fre_samp, endpoint = False )
a = np.sin(fre  * 2 * np.pi * t)
figure, axis = plt.subplots()
axis.plot(t, a)
axis.set_xlabel ('Time (s)')
axis.set_ylabel ('Signal amplitude')
plt.show()
```



logictronix